

RNBOPatch() for p5js

Download Library/Template

To get started with the RNBOPatch() object first download the template from [here](#) and make sure you have the most recent version of the processing editor for p5js from [here](#).

Step 1: Getting the most recent version of RNBO for js

The RNBOPatch() p5js library is built off of the cycling74 RNBO javascript library so the first thing that needs to be done is adding this line to your index.html file to pull the most updated version of that library from the web:

```
<script language="javascript" type="text/javascript"
src="https://cdn.cycling74.com/rnbo/latest/rnbo.min.js"></script>
```

It should go on the line directly between these two autogenerated lines

```
<script language="javascript" type="text/javascript" src="RNBOp5lib.js"></script>
<- - - New line goes here
<script language="javascript" type="text/javascript" src="NAMEOFYOURSKETCH.js"></script>
```

The p5js editor will regenerate this html file every time you add a new tab or change the name of an existing tab in your program so if you run into errors there is a chance you will need to readd this line again as it's been automatically deleted.

In addition to the html file you will need to include the following line at the top of your main tab of code containing the draw() and setup() functions. This allows your code to actually see the newest version of RNBO that was imported in the html.

```
const { createDevice } = RNBO;
```

Step 2: Getting the most recent version of the RNBO for p5js library

If you are using the template you can skip this step. If you are not using the template after creating your p5js file you will need to copy the file titled "RNBOp5lib.js" and the folder titled "patches" into your project folder.

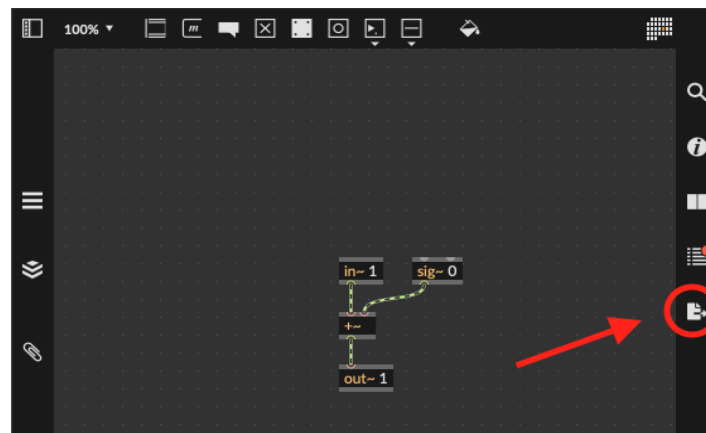
**** Double check that the patches folder contains the file "inpComp.export.json" otherwise the library will crash****

Your file directory should look like this

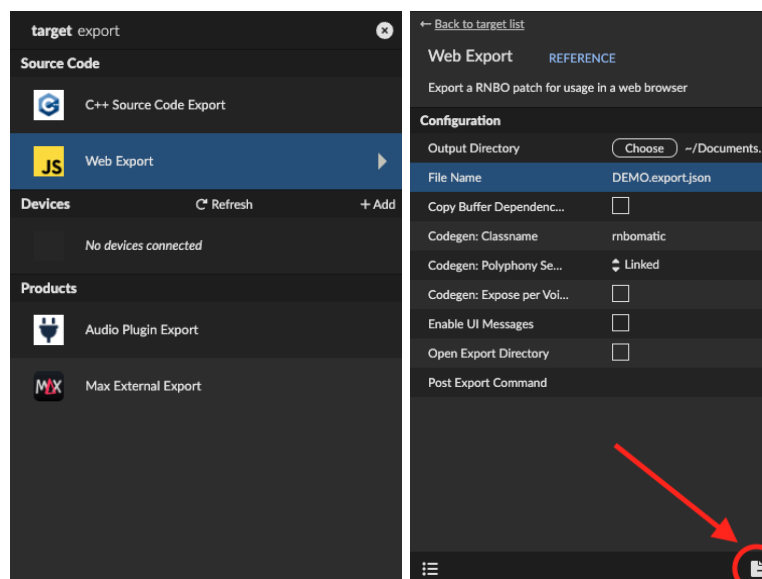
```
YOURPATCH.js
RNBOp5lib.js
index.html
sketch.properties
libraries(Folder)
  > p5.min.js
patches(Folder)
  > inpComp.export.json
```

Step 3: Getting your patch(or patches) from Max

For this documentation I will use this simple buffer patch. Start by pressing the “Show Export Sidebar” button shown in the figure below.



From there select js Web Export from the options and then set your output directory to be the patches folder of your p5js program. Set the file name. The majority of RNBO patches won't need any of the other options selected so try exporting as is and if there's issues consult the guide on exporting RNBO from Cycling74 on what options may need to be selected. When you're done press export.



This will not only generate the json file version of your patch but also a dependencies file and a license file. You can get rid of the license file to avoid overwriting issues same with the dependencies file IF IT IS EMPTY OTHER THAN THESE BRACKETS []. If the file is not empty move your patch and the dependencies into a new subfolder within patches so you don't overwrite the dependencies if you are working with multiple RNBO patch exports.

Do this for as many RNBO patches you need for your p5js projects. Theoretically you can do unlimited with this library but there will eventually be a throttle based on what the web browser can handle.

Step 4: Asynchronous audioSetup() function/draw() setup

If you are using the template you can also skip this step but it may be helpful to review just to understand what's going on. Since we need to wait for the RNBO patches to load we need to use an async function to setup the creation of RNBO patch objects. In javascript using a asynchronous function allows you to use the await keyword to wait for a function to finish before continuing the code.

You will want to create an asynchronous function in the top of your main code tab with the draw() and setup() functions. I also recommend creating a variable to tell when your async function is done and in this case as well as the template I will call it rnboloading.

```
async function audioSetup() {
  audioContextSetup();
  example = new RNBOPatch('patches/NAMEOFOURPATCH.export.json');
  await example.init();
  rnboloading = true;
}
```

audioContextSetup() is called to begin the webkit or non webkit(for older browsers) audio context which is needed to have audio involved in your site.

Every time you create a RNBOPatch object you must feed it the argument of the patch location. You must also then run the .init() function with the await keyword to actually tether your RNBO export to this object. Create and init as many RNBOPatch() objects in this function as you need. You can also create multiple objects using the same export file and run into no issues. This is the place to toggle your rnboloading variable to true.

For every patch object variable like example in this case you will need to define it at the top of your program under the line where we imported the js RNBO device. The top of the tab should look like this:

```
const { createDevice } = RNBO;
let rnboloading = false;
let example;
```

Once this function is fully written you can call it inside of the the main setup() function like this:

```
function setup() {
  let canvas = createCanvas(windowWidth, windowHeight);
  audioSetup();
}
```

The only setup left is to call the startAudioContext() function within your draw function to repeatedly check that the browser audio is running. You can then write your main website code after checking that rnboloading is true.

```
function draw() {
  startAudioContext();
  if (rnboloading) {
    //Your code here
  }
}
```

RNBOPatch() Methods/Constructors

constructor: RNBOPatch(String fileLocation)

input: a string containing the filepath of the RNBO export .json

output: RNBOPatch object

Method Name	Input	Output	Notes
init()	n/a	n/a	Must be called with the await keyword i.e. await example.init();
connect()	RNBOPatchInput in, int outIndex	n/a	The out index will be the number on the out~ from Max minus 1 for standard array indexing. The RNBOPatchInput object can only be obtained through the getInput() function of a RNBOPatch() object
disconnect()	int outIndex	n/a	Read above for indexing
disconnectAll()	n/a	n/a	Disconnects all out~
setParam()	String paramName, float or int newValue	n/a	The paramName string must be exactly as the param is written in the patch in Max. newValue can be a float or an int depending on whatever the param is set to take
numOutputs()	n/a	number of outputs the object has as an int	n/a
numInputs()	n/a	number of inputs the object has as an int	n/a
getInput()	int index	RNBOPatchInput object	Index of in~ object -1 to align with standard array indexing
masterOut()	n/a	n/a	Sends a 2 channel stereo out to the web browser audio directly. Use this for the last RNBO patch in your chain.